

Comportamiento del cerebro humano.

Primer código:

```
"""
```

```
=====
```

```
CLAVE - Cerebro Artificial Modular  
Arquitectura inspirada en neurociencia
```

```
=====
```

```
"""
```

```
import numpy as np  
from collections import deque  
import random
```

```
# -----
```

```
# MÓDULO 1: LA NEURONA FUNDAMENTAL DE CLAVE
```

```
# El "ciudadano" básico. Todo lo demás se construye con esto.
```

```
# -----
```

```
class NeuronaClave:
```

```
    """
```

```
    Simplifica la neurona biológica:
```

- Dendritas: Reciben señales ponderadas (entradas * pesos)
- Soma: Suma las señales (suma ponderada + bias)
- Axón: Aplica función de activación y transmite

```
    """
```

```
    def __init__(self, num_entradas, tipo_activacion='relu'):
```

```
        # "Sinapsis": Pesos que representan la fuerza de cada conexión
```

```
        self.pesos = np.random.randn(num_entradas) * np.sqrt(2.0 /  
num_entradas)
```

```
        self.bias = 0.0
```

```
        self.tipo_activacion = tipo_activacion
```

```
        # Para plasticidad (aprendizaje)
```

```
        self.grad_pesos = np.zeros_like(self.pesos)
```

```
        self.grad_bias = 0.0
```

```

def activar(self, entrada):
    """El potencial de acción: suma ponderada -> activación"""
    self.entrada = entrada
    self.suma_ponderada = np.dot(entrada, self.pesos) + self.bias

    if self.tipo_activacion == 'relu':
        self.salida = max(0, self.suma_ponderada)
    elif self.tipo_activacion == 'sigmoide':
        self.salida = 1.0 / (1.0 + np.exp(-self.suma_ponderada))

    return self.salida

```

```

# -----
# MÓDULO 2: CAPA DE NEURONAS (Columna Cortical Simplificada)
# Un "minicolumna" como las de la corteza cerebral.
# -----
class CapaCortical:
    """Una capa de neuronas que opera como una unidad de
    procesamiento"""
    def __init__(self, num_entradas, num_neuronas, tipo='oculta'):
        self.neuronas = [NeuronaClave(num_entradas) for _ in
        range(num_neuronas)]
        self.tipo = tipo
        self.salida = np.zeros(num_neuronas)

    def forward(self, entrada):
        """Propagación hacia adelante por toda la capa"""
        self.salida = np.array([neurona.activar(entrada) for neurona in
        self.neuronas])
        return self.salida

    def actualizar_pesos(self, tasa_aprendizaje):
        """Plasticidad: ajusta los pesos según el error calculado"""

```

```
for neurona in self.neuronas:
    neurona.pesos -= tasa_aprendizaje * neurona.grad_pesos
    neurona.bias -= tasa_aprendizaje * neurona.grad_bias
```

```
# -----
# MÓDULO 3: LÓBULO OCCIPITAL (VISIÓN SIMPLIFICADA)
# No procesa píxeles, sino "características visuales" pre-extraídas
# -----
class LobuloOccipital:
    """
    Simula la corteza visual jerárquica:
    V1 -> V2 -> V4 -> IT (reconocimiento de objetos)
    Simplificado a 3 capas que extraen características progresivamente.
    """
    def __init__(self, dim_entrada=100, dim_salida=32):
        # V1-V2: Detectores de bordes y formas simples
        self.capa_v1 = CapaCortical(dim_entrada, 64, 'visual_temprana')
        # V4: Formas complejas y color
        self.capa_v4 = CapaCortical(64, 48, 'visual_media')
        # IT: Reconocimiento de objetos (representación compacta)
        self.capa_it = CapaCortical(48, dim_salida, 'visual_superior')

        # Memoria de patrones visuales conocidos
        self.memoria_visual = {}

    def ver(self, estimulo_visual):
        """
        Procesa un estímulo visual a través de la jerarquía.
        entrada: vector de características visuales (ej. de una CNN
        pre-entrenada)
        salida: representación abstracta del objeto
        """
        # Jerarquía visual
        v1_out = self.capa_v1.forward(estimulo_visual)
```

```

v4_out = self.capa_v4.forward(v1_out)
percepcion_visual = self.capa_it.forward(v4_out)

return percepcion_visual

def reconocer_objeto(self, estimulo_visual, umbral=0.8):
    """Reconoce si el objeto es conocido (como la corteza
inferotemporal)"""
    percepcion = self.ver(estimulo_visual)

    for nombre_objeto, representacion in self.memoria_visual.items():
        similitud = self._calcular_similitud(percepcion, representacion)
        if similitud > umbral:
            return nombre_objeto, similitud

    return "desconocido", 0.0

def aprender_objeto(self, nombre, estimulo_visual):
    """Memoriza un nuevo objeto (plasticidad en IT)"""
    representacion = self.ver(estimulo_visual)
    self.memoria_visual[nombre] = representacion
    print(f"[Occipital] He aprendido a reconocer: {nombre}")

def _calcular_similitud(self, vec1, vec2):
    """Similitud coseno como medida de reconocimiento"""
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) *
np.linalg.norm(vec2) + 1e-8)

# -----
# MÓDULO 4: LÓBULO TEMPORAL (AUDICIÓN Y LENGUAJE)
# Procesamiento de secuencias y comprensión semántica
# -----
class LobuloTemporal:
    """

```

Simula el procesamiento auditivo y del lenguaje:

- Corteza auditiva: Procesa sonidos
- Área de Wernicke: Comprensión del lenguaje

"""

```
def __init__(self, dim_entrada=50, dim_salida=32):
```

```
    # Corteza auditiva primaria
```

```
    self.capa_auditiva = CapaCortical(dim_entrada, 48, 'auditiva')
```

```
    # Área de Wernicke: significado semántico
```

```
    self.area_wernicke = CapaCortical(48, dim_salida, 'semantica')
```

```
    # Memoria semántica: conceptos y sus relaciones
```

```
    self.red_semantica = {}
```

```
    self.memoria_secuencial = deque(maxlen=10) # Memoria ecoica a  
corto plazo
```

```
def escuchar(self, entrada_auditiva):
```

```
    """
```

```
    Procesa entrada auditiva (ya pre-procesada a características)
```

```
    retorna: representación semántica
```

```
    """
```

```
    # Procesamiento auditivo
```

```
    activacion_auditiva = self.capa_auditiva.forward(entrada_auditiva)
```

```
    # Comprensión semántica
```

```
    significado = self.area_wernicke.forward(activacion_auditiva)
```

```
    # Guardar en memoria ecoica (a corto plazo)
```

```
    self.memoria_secuencial.append(significado)
```

```
    return significado
```

```
def comprender_contexto_temporal(self):
```

```
    """
```

```
    Integra las últimas entradas para entender el contexto.
```

```
    Como el área de Wernicke procesando una frase completa.
```

```
    """
```

```

if len(self.memoria_secuencial) < 2:
    return self.memoria_secuencial[-1] if self.memoria_secuencial else
np.zeros(32)

```

```

# Promedio ponderado: más peso a lo reciente
contexto = np.zeros(32)
for i, mem in enumerate(self.memoria_secuencial):
    peso = (i + 1) / len(self.memoria_secuencial)
    contexto += mem * peso

```

```

return contexto / len(self.memoria_secuencial)

```

```

def asociar_concepto(self, palabra, significado_vector):
    """Aprendizaje semántico: asocia palabra con representación"""
    self.red_semantica[palabra] = significado_vector
    print(f"[Temporal] Concepto '{palabra}' integrado en la red semántica")

```

```

# -----

```

```

# MÓDULO 5: AMÍGDALA (SISTEMA EMOCIONAL Y DE ALERTA)

```

```

# Evaluación rápida de relevancia emocional

```

```

# -----

```

```

class Amigdala:

```

```

    """

```

```

    Sistema de evaluación emocional rápida:

```

```

    - Vía rápida (tálamo->amígdala): Respuesta inmediata

```

```

    - Vía lenta (tálamo->corteza->amígdala): Evaluación refinada

```

```

    """

```

```

    def __init__(self, dim_entrada=32):

```

```

        # Evaluador rápido (poco preciso, pero veloz)

```

```

        self.evaluador_rapido = CapaCortical(dim_entrada, 8,
'emocional_rapida')

```

```

        self.capa_valencia = CapaCortical(8, 1, 'valencia')

```

```

        # Memoria emocional: eventos con carga emocional

```

```

self.memoria_emocional = []
self.estado_actual = {
    'valencia': 0.0,    # -1 (miedo/desagrado) a +1 (placer)
    'activacion': 0.0,  # 0 (calma) a 1 (alerta máxima)
    'dominancia': 0.5   # 0 (sin control) a 1 (control total)
}

def evaluar_situacion(self, estimulo):
    """
    Evalúa la carga emocional de un estímulo.
    Como la amígdala basolateral evaluando una situación.
    """
    # Vía rápida: evaluación inmediata
    respuesta_rapida = self.evaluador_rapido.forward(estimulo)
    valencia_cruda = self.capa_valencia.forward(respuesta_rapida)[0]

    # Normalizar entre -1 y 1
    valencia = np.tanh(valencia_cruda)

    # Actualizar estado emocional
    self.estado_actual['valencia'] = valencia
    self.estado_actual['activacion'] = abs(valencia) # Intensidad = valor
    absoluto

    # Si es muy intenso, guardar en memoria emocional
    if abs(valencia) > 0.6:
        self._marcar_como_significativo(estimulo, valencia)

    return valencia

def _marcar_como_significativo(self, estimulo, valencia):
    """Consolidación de memoria emocional"""
    self.memoria_emocional.append({
        'estimulo': estimulo.copy(),
        'valencia': valencia,

```

```

        'intensidad': abs(valencia)
    })
    # Mantener solo los últimos 20 recuerdos emocionales
    if len(self.memoria_emocional) > 20:
        self.memoria_emocional.pop(0)

def consultar_estado(self):
    """Retorna el estado emocional actual"""
    return self.estado_actual

def recordar_evento_similar(self, estimulo, umbral=0.7):
    """Busca en la memoria emocional eventos parecidos"""
    for recuerdo in self.memoria_emocional:
        similitud = np.dot(estimulo, recuerdo['estimulo']) / (
            np.linalg.norm(estimulo) * np.linalg.norm(recuerdo['estimulo']) +
1e-8
        )
        if similitud > umbral:
            return recuerdo
    return None

```

```

# -----
# MÓDULO 6: HIPOCAMPO (SISTEMA DE MEMORIA EPISÓDICA)
# Formación y recuperación de recuerdos
# -----

class Hipocampo:
    """
    Simula el sistema de memoria del hipocampo:
    - Codificación: Convierte experiencias en "huellas de memoria"
    - Consolidación: Fortalece recuerdos importantes
    - Recuperación: Recuerda por similitud contextual
    """

    def __init__(self, dim_representacion=32):
        self.dim_representacion = dim_representacion

```

```
self.memoria_episodica = [] # Almacén de episodios
self.memoria_trabajo = deque(maxlen=7) # Mágico número  $7 \pm 2$  de
Miller
```

```
self.indice_consolidacion = 0
```

```
def codificar_episodio(self, contexto, emocion, accion, resultado):
```

```
    """
```

```
    Codifica una experiencia completa.
```

```
    Como el giro dentado creando una nueva representación.
```

```
    """
```

```
    episodio = {
```

```
        'id': self.indice_consolidacion,
```

```
        'contexto': contexto.copy(),      # Dónde y qué
```

```
        'emocion': emocion,              # Cómo me sentí
```

```
        'accion': accion,                 # Qué hice
```

```
        'resultado': resultado,           # Qué pasó después
```

```
        'fuerza': abs(emocion) * 0.5,     # La emoción modula la fuerza
```

```
        'accesos': 1,                    # Cada recuerdo fortalece el recuerdo
```

```
        'timestamp': self.indice_consolidacion
```

```
    }
```

```
self.memoria_episodica.append(episodio)
```

```
self.indice_consolidacion += 1
```

```
# Consolidación: mantener un límite práctico
```

```
if len(self.memoria_episodica) > 100:
```

```
    self._consolidar_memorias()
```

```
    print(f"[Hipocampo] Episodio #{episodio['id']} codificado (fuerza:
{episodio['fuerza']:.2f})")
```

```
    return episodio['id']
```

```
def recuperar_por_contexto(self, contexto_actual, k=3):
```

```
    """
```

```
    Recupera los k recuerdos más similares al contexto actual.
```

Como la recuperación por patrones de finalización en CA3.

```
"""
if not self.memoria_episodica:
    return []

similitudes = []
for episodio in self.memoria_episodica:
    sim = np.dot(contexto_actual, episodio['contexto']) / (
        np.linalg.norm(contexto_actual) *
np.linalg.norm(episodio['contexto']) + 1e-8
    )
    # Ponderar por fuerza del recuerdo
    puntuacion = sim * episodio['fuerza']
    similitudes.append((puntuacion, episodio))

# Ordenar y retornar los k mejores
similitudes.sort(key=lambda x: x[0], reverse=True)
recuperados = [ep for _, ep in similitudes[:k]]

# Fortalecer recuerdos accedidos (reconsolidación)
for ep in recuperados:
    ep['fuerza'] *= 1.1
    ep['accesos'] += 1

return recuperados

def _consolidar_memorias(self):
    """
    Elimina recuerdos débiles, fortalece los importantes.
    Como la consolidación durante el sueño.
    """
    # Ordenar por fuerza * accesos
    self.memoria_episodica.sort(
        key=lambda ep: ep['fuerza'] * (1 + 0.1 * ep['accesos']),
        reverse=True
```

```
)
# Mantener solo los más fuertes (los 50 principales)
self.memoria_episodica = self.memoria_episodica[:50]
print("[Hipocampo] Consolidación completada: memorias débiles
eliminadas")
```

```
# -----
# MÓDULO 7: CORTEZA PREFRONTAL (FUNCIONES EJECUTIVAS)
# El "Director General" de Clave
# -----
```

```
class CortezaPrefrontal:
```

```
    """
```

```
    Simula las funciones ejecutivas:
```

- Memoria de trabajo
- Planificación
- Control inhibitorio
- Toma de decisiones

```
    """
```

```
    def __init__(self, dim_entrada=96): # 32*3 (visual + semántico +
emocional)
```

```
        self.dim_entrada = dim_entrada
```

```
        self.memoria_trabajo = deque(maxlen=7)
```

```
        self.objetivos_activos = []
```

```
        self.plan_actual = None
```

```
        # Red de decisión: evalúa opciones y elige acciones
```

```
        self.capa_evaluacion = CapaCortical(dim_entrada, 64,
'prefrontal_eval')
```

```
        self.capa_decision = CapaCortical(64, 8, 'prefrontal_dec')
```

```
        # Sistema de recompensa y error
```

```
        self.historial_decisiones = []
```

```

def integrar_informacion(self, percepcion_visual,
comprension_semantica, estado_emocional):
    """
    Integra toda la información como la corteza prefrontal.
    Esta es la "conciencia de trabajo" de Clave.
    """
    # Concatenar toda la información sensorial y emocional
    informacion_integrada = np.concatenate([
        percepcion_visual,
        comprension_semantica,
        np.array([estado_emocional['valencia'],
                    estado_emocional['activacion'],
                    estado_emocional['dominancia']])
    ])

    # Ajustar tamaño si es necesario
    if len(informacion_integrada) < self.dim_entrada:
        informacion_integrada = np.pad(informacion_integrada,
                                        (0, self.dim_entrada -
len(informacion_integrada)))

    self.memoria_trabajo.append(informacion_integrada)
    return informacion_integrada

def evaluar_opciones(self, opciones, contexto_integrado):
    """
    Evalúa diferentes cursos de acción posibles.
    Como la corteza orbitofrontal evaluando recompensas.
    """
    evaluaciones = []
    for opcion in opciones:
        # Combinar contexto con opción
        entrada_eval = np.concatenate([contexto_integrado[:64],
opcion[:32]])
        if len(entrada_eval) < self.dim_entrada:

```

```
        entrada_eval = np.pad(entrada_eval, (0, self.dim_entrada -
len(entrada_eval)))
```

```
        valor =
self.capa_evaluacion.forward(entrada_eval[:self.dim_entrada])
        valor_final = self.capa_decision.forward(valor)
        evaluaciones.append(np.mean(valor_final))
```

```
    return evaluaciones
```

```
def tomar_decision(self, opciones, contexto_integrado):
```

```
    """
```

```
    Elige la mejor acción basada en evaluación y estado actual.
```

```
    """
```

```
    if not opciones:
```

```
        return None, 0.0
```

```
    evaluaciones = self.evaluar_opciones(opciones, contexto_integrado)
```

```
    mejor_idx = np.argmax(evaluaciones)
```

```
    decision = {
```

```
        'opcion_elegida': mejor_idx,
```

```
        'confianza': evaluaciones[mejor_idx],
```

```
        'alternativas': evaluaciones,
```

```
        'contexto': contexto_integrado.copy()
```

```
    }
```

```
    self.historial_decisiones.append(decision)
```

```
    print(f"[Prefrontal] Decisión tomada: Opción {mejor_idx} (confianza:
{evaluaciones[mejor_idx]:.2f})")
```

```
    return mejor_idx, evaluaciones[mejor_idx]
```

```
def establecer_objetivo(self, objetivo):
```

```

        """Mantiene un objetivo en mente (como la corteza prefrontal
dorsolateral)"""
        self.objetivos_activos.append(objetivo)
        print(f"[Prefrontal] Nuevo objetivo establecido: {objetivo}")

def inhibir_respuesta(self, respuesta_automatica, contexto):
    """
    Control inhibitorio: frena respuestas inapropiadas.
    Como la corteza prefrontal ventromedial.
    """
    # Si hay conflicto entre respuesta automática y objetivos
    if self.objetivos_activos and
self._conflicto_con_objetivos(respuesta_automatica):
        print("[Prefrontal] Respuesta inhibida por conflicto con objetivos")
        return True
    return False

def _conflicto_con_objetivos(self, respuesta):
    """Verifica si una respuesta contradice los objetivos activos"""
    # Simplificado: en una implementación real, esto sería más sofisticado
    return random.random() < 0.3 # Probabilístico para demostración

# -----
# MÓDULO 8: CEREBELO (COORDINACIÓN Y REFINAMIENTO)
# Aprendizaje motor y ajuste fino de acciones
# -----
class Cerebelo:
    """
    Simula el cerebelo como refinador de acciones:
    - No inicia acciones, las pule
    - Aprendizaje supervisado de errores motores
    """
    def __init__(self, dim_accion=10):
        self.dim_accion = dim_accion

```

```

        self.modelo_interno = CapaCortical(dim_accion * 2, dim_accion,
'cerebelo')
        self.error_acumulado = 0

def refinar_accion(self, accion_bruta, contexto_motor):
    """
    Refina una acción motora.
    Como el cerebelo ajustando movimientos basado en experiencia.
    """
    entrada = np.concatenate([accion_bruta, contexto_motor])
    if len(entrada) < 20:
        entrada = np.pad(entrada, (0, 20 - len(entrada)))

    correccion = self.modelo_interno.forward(entrada[:20])
    accion_refinada = accion_bruta + 0.1 * correccion

    return accion_refinada

def aprender_de_error(self, accion_prevista, resultado_real):
    """
    Aprendizaje por error: como las fibras trepadoras
    señalando errores de predicción.
    """
    error = resultado_real - accion_prevista
    self.error_acumulado += np.mean(np.abs(error))
    print(f"[Cerebelo] Error detectado: {np.mean(np.abs(error)):.3f}")

```

```

# -----
# MÓDULO 9: TRONCO ENCEFÁLICO Y HOMEOSTASIS
# Regulación automática vital
# -----
class TroncoEncefalico:
    """
    Sistema homeostático que mantiene el equilibrio interno.

```

Como el hipotálamo y tronco encefálico.

"""

```
def __init__(self):
```

```
    self.necesidades = {
```

```
        'energia': 1.0,    # 0 = agotado, 1 = lleno
```

```
        'seguridad': 1.0,  # 0 = amenaza, 1 = seguro
```

```
        'curiosidad': 0.5, # Deseo de explorar
```

```
        'descanso': 1.0    # 0 = agotado, 1 = descansado
```

```
    }
```

```
    self.tasa_decaimiento = {
```

```
        'energia': 0.02,
```

```
        'seguridad': 0.01,
```

```
        'curiosidad': 0.005,
```

```
        'descanso': 0.015
```

```
    }
```

```
def actualizar_homeostasis(self):
```

```
    """Actualiza las necesidades internas con el tiempo"""
```

```
    for necesidad in self.necesidades:
```

```
        self.necesidades[necesidad] -= self.tasa_decaimiento[necesidad]
```

```
        self.necesidades[necesidad] = max(0, self.necesidades[necesidad])
```

```
def generar_impulso(self):
```

```
    """
```

```
    Genera impulsos básicos basados en necesidades.
```

```
    Como el hipotálamo generando señales de hambre/sed.
```

```
    """
```

```
    impulsos = {}
```

```
    if self.necesidades['energia'] < 0.3:
```

```
        impulsos['buscar_recurso'] = 1 - self.necesidades['energia']
```

```
    if self.necesidades['seguridad'] < 0.3:
```

```
        impulsos['buscar_seguridad'] = 1 - self.necesidades['seguridad']
```

```
    if self.necesidades['curiosidad'] > 0.6:
```

```
        impulsos['explorar'] = self.necesidades['curiosidad']
```

```
    if self.necesidades['descanso'] < 0.2:
```

```

        impulsos['descansar'] = 1 - self.necesidades['descanso']

    return impulsos

def satisfacer_necesidad(self, necesidad, cantidad=0.5):
    """Satisface una necesidad específica"""
    if necesidad in self.necesidades:
        self.necesidades[necesidad] += cantidad
        self.necesidades[necesidad] = min(1.0,
self.necesidades[necesidad])
        print(f"[TroncoEncefalico] Necesidad '{necesidad}' satisfecha
(+{cantidad})")

# -----
# MÓDULO 10: SISTEMA DE ATENCIÓN (SARA)
# Sistema Activador Reticular Ascendente
# -----
class SistemaAtencion:
    """
    Controla el foco atencional y el estado de alerta.
    """
    def __init__(self, dim_entrada=96):
        self.foco_actual = None
        self.nivel_alerta = 0.5 # 0=dormido, 1=hiperalerta
        self.filtro_atencional = CapaCortical(dim_entrada, dim_entrada,
'atencion')

    def enfocar_atencion(self, informacion_entrada, prioridades):
        """
        Filtra la información según relevancia.
        Como el SARA filtrando estímulos irrelevantes.
        """
        # Combinar información con prioridades
        entrada_filtrada = self.filtro_atencional.forward(informacion_entrada)

```

```

# Si hay prioridad de seguridad, aumentar alerta
if prioridades and 'seguridad' in prioridades:
    self.nivel_alerta = min(1.0, self.nivel_alerta + 0.1)

return entrada_filtrada * self.nivel_alerta

```

```

#
=====
=
# "CLAVE" - EL CEREBRO COMPLETO INTEGRADO
# Aquí todas las partes trabajan juntas
#
=====
=
class Clave:
    """
    Arquitectura completa de cerebro artificial.
    Inspirada en la organización funcional del cerebro humano.
    """
    def __init__(self, nombre="Clave"):
        self.nombre = nombre
        self.edad_mental = 0 # Ciclos de experiencia

        # Todos los módulos trabajando en conjunto
        self.occipital = LobuloOccipital(dim_entrada=100, dim_salida=32)
        self.temporal = LobuloTemporal(dim_entrada=50, dim_salida=32)
        self.amigdala = Amigdala(dim_entrada=32)
        self.hipocampo = Hipocampo(dim_representacion=32)
        self.prefrontal = CortezaPrefrontal(dim_entrada=96)
        self.cerebelo = Cerebelo(dim_accion=10)
        self.tronco = TroncoEncefalico()
        self.atencion = SistemaAtencion(dim_entrada=96)

```

```

# Estado de conciencia (simplificado)
self.consciente = True
self.memoria_procedimental = {} # Hábitos y habilidades

print(f"""
    {self.nombre} - Cerebro Artificial    ||
    Inspirado en neurociencia          ||
""")

def percibir_mundo(self, estimulo_visual, estimulo_auditivo=None):
    """
    Percibe el mundo a través de los sentidos.
    Entrada principal de información a Clave.
    """
    # Procesamiento paralelo (como en el cerebro real)
    percepcion_visual = self.occipital.ver(estimulo_visual)

    if estimulo_auditivo is not None:
        comprension_semantica =
self.temporal.escuchar(estimulo_auditivo)
    else:
        comprension_semantica = np.zeros(32)

    return percepcion_visual, comprension_semantica

def sentir(self, estimulo_interno=None):
    """
    Genera el estado emocional basado en estímulos.
    """
    # Evaluación emocional de la situación
    if estimulo_interno is None:
        estimulo_interno = np.random.randn(32) * 0.1

```

```

    valencia = self.amigdala.evaluar_situacion(estimulo_interno)
    estado = self.amigdala.consultar_estado()

    return estado, valencia

def pensar(self, percepcion_visual, comprension_semantica,
estado_emocional):
    """
    Integra información y toma decisiones.
    Este es el "espacio de trabajo global" de Clave.
    """
    # Integración multimodal (como la corteza prefrontal)
    contexto_integrado = self.prefrontal.integrar_informacion(
        percepcion_visual, comprension_semantica, estado_emocional
    )

    # Recuperar memorias relevantes para informar la decisión
    recuerdos =
self.hipocampo.recuperar_por_contexto(contexto_integrado[:32])

    # Generar opciones de acción basadas en necesidades y recuerdos
    impulsos = self.tronco.generar_impulso()
    opciones = self._generar_opciones(impulsos, recuerdos,
estado_emocional)

    # Tomar decisión
    accion_idx, confianza = self.prefrontal.tomar_decision(opciones,
contexto_integrado)

    return accion_idx, confianza, recuerdos, contexto_integrado

def actuar(self, accion_idx, contexto):
    """
    Ejecuta una acción refinada por el cerebelo.
    """

```

```

# Acción base
accion_base = np.zeros(10)
accion_base[accion_idx] = 1.0

# El cerebelo refina la acción
contexto_motor = contexto[:10] if len(contexto) >= 10 else
np.pad(contexto, (0, 10-len(contexto)))
accion_refinada = self.cerebelo.refinar_accion(accion_base,
contexto_motor)

# Verificar si debemos inhibir la acción (control prefrontal)
if self.prefrontal.inhibir_respuesta(accion_refinada, contexto):
    print(f"[{self.nombre}] Acción inhibida por control ejecutivo")
    return None

return accion_refinada

def aprender_experiencia(self, percepcion, emocion, accion, resultado):
    """
    Aprende de la experiencia (ciclo completo de aprendizaje).
    """
    # 1. Codificar en memoria episódica (hipocampo)
    self.hipocampo.codificar_episodio(percepcion, emocion, accion,
resultado)

    # 2. Si hay error, el cerebelo aprende
    if resultado < 0.5:
        self.cerebelo.aprender_de_error(accion, resultado)

    # 3. Actualizar homeostasis
    self.tronco.actualizar_homeostasis()

    self.edad_mental += 1

def _generar_opciones(self, impulsos, recuerdos, estado_emocional):

```

```

"""Genera posibles acciones basadas en estado interno y memorias"""
opciones = []

# Opciones basadas en impulsos
for impulso, intensidad in impulsos.items():
    opcion = np.random.randn(96) * intensidad
    opciones.append(opcion)

# Si no hay impulsos, generar opciones por defecto
if not opciones:
    opciones = [np.random.randn(96) for _ in range(3)]

return opciones

def ciclo_completo(self, estimulo_visual, estimulo_auditivo=None):
    """
    Un ciclo completo de percepción-pensamiento-acción de Clave.
    """
    print(f"\n{'='*50}")
    print(f"[{self.nombre}] Ciclo mental #{self.edad_mental + 1}")
    print(f"\n{'='*50}")

    # 1. PERCIBIR
    print("\n[1] PERCIBIENDO...")
    vis, sem = self.percibir_mundo(estimulo_visual, estimulo_auditivo)
    print(f"  Visión procesada: {vis[:4]}...")

    # 2. SENTIR
    print("\n[2] SINTIENDO...")
    estado_emocional, valencia = self.sentir()
    print(f"  Estado emocional:
valencia={estado_emocional['valencia']:.2f}, "
        f"activación={estado_emocional['activacion']:.2f}")

    # 3. PENSAR Y DECIDIR

```

```

        print("\n[3] PENSANDO...")
        accion_idx, confianza, recuerdos, contexto = self.pensar(vis, sem,
estado_emocional)
        print(f"    Recuerdos recuperados: {len(recuerdos)}")

```

```

# 4. ACTUAR
print("\n[4] ACTUANDO...")
accion = self.actuar(accion_idx, contexto)
if accion is not None:
    print(f"    Acción ejecutada: {accion_idx} (confianza:
{confianza:.2f})")

```

```

# 5. APRENDER (simulando un resultado)
print("\n[5] APRENDIENDO...")
resultado_simulado = random.uniform(0.3, 0.9)
self.aprender_experiencia(vis[:32], valencia, accion_idx,
resultado_simulado)

```

```

return {
    'percepcion': vis,
    'emocion': estado_emocional,
    'decision': accion_idx,
    'confianza': confianza,
    'resultado': resultado_simulado
}

```

```

def reporte_estado(self):
    """Genera un reporte del estado interno de Clave"""
    print(f"""

```

```

    ESTADO INTERNO DE {self.nombre}    ||
    ||-----||
    || Edad mental: {self.edad_mental} ciclos    ||
    || Memorias episódicas: {len(self.hipocampo.memoria_episodica)}
||

```

```

|| Objetos reconocidos: {len(self.occipital.memoria_visual)} ||
|| Conceptos semánticos: {len(self.temporal.red_semantica)} ||
||-----||
|| Necesidades: ||
|| Energía: {self.tronco.necesidades['energia']:.2f} ||
|| Seguridad: {self.tronco.necesidades['seguridad']:.2f} ||
|| Curiosidad: {self.tronco.necesidades['curiosidad']:.2f} ||
|| Descanso: {self.tronco.necesidades['descanso']:.2f} ||
||-----||
""")

```

```

#
=====
=
# DEMOSTRACIÓN: CLAVE EN ACCIÓN
#
=====
=
if __name__ == "__main__":
    # Crear a Clave
    clave = Clave(nombre="Clave v1.0")

    # Enseñarle a reconocer objetos (aprendizaje visual)
    print("\n[ENTRENAMIENTO] Enseñando a Clave a reconocer objetos...")
    for _ in range(5):
        objeto_aleatorio = np.random.randn(100)
        clave.occipital.aprender_objeto(f"objeto_{_}", objeto_aleatorio)

    # Enseñarle conceptos semánticos
    print("\n[ENTRENAMIENTO] Enseñando conceptos semánticos...")
    conceptos = ['peligro', 'seguro', 'interesante', 'aburrido']
    for concepto in conceptos:
        clave.temporal.asociar_concepto(concepto, np.random.randn(32))

```

```

# Ejecutar varios ciclos de Clave
print("\n" + "="*60)
print("CLAVE EN FUNCIONAMIENTO")
print("="*60)

for ciclo in range(3):
    # Estímulos aleatorios simulados
    estimulo_visual = np.random.randn(100)
    estimulo_auditivo = np.random.randn(50)

    resultado = clave.ciclo_completo(estimulo_visual, estimulo_auditivo)

    # Mostrar estado cada 2 ciclos
    if ciclo % 2 == 0:
        clave.reporte_estado()

print(f"\n[FIN] Clave ha completado {clave.edad_mental} ciclos de
experiencia.")
print("Arquitectura neuronal funcionando de forma integrada.")

```

La recreación del cerebro humano en el lenguaje computacional de la informática es momentáneamente imposible, tal cual que una AGI (Inteligencia Artificial General (inteligencia artificial REAL)) es momentáneamente imposible, ficticia, debido a la capacidad de la computación de hoy en día, apenas se pudo computarizar el cerebro de una pequeña mosca en una supercomputadora.

Intentar hacer lo mismo con el cerebro humano equivaldría a usar aproximadamente el 100% de la capacidad de más de 100 supercomputadoras simultáneamente para poder recrear el cerebro humano estable en un programa, por eso se dice que es ficticio, ya que esa sería una AGI real, una IA independiente y general en todo aspecto relacionado al comportamiento cerebral.

Pero acá me la pela todo así que voy a buscar y rebuscar la forma en crear la primera AGI (que conozco) llamada C.L.A.V.E, o Clave, las siglas no significan nada particular, así que reiniciando nuevamente el código de clave, voy a estar subiendo versiones para que veas que laburo, mientras limpio, encero, pulo, afilo, y filtro la verdadera AGI.

(yo te dije que las IA no son Inteligencia Artificial Real).

""""

=====

=====

CLAVE v3.0 - Cerebro Artificial Modular, Persistente e Interactivo

=====

=====

Este archivo unifica dos versiones previas de CLAVE:

- CLAVE v1.0: arquitectura biológica completa (lóbulo occipital, lóbulo temporal, amígdala, hipocampo, corteza prefrontal, cerebelo, tronco encefálico y sistema de atención) organizada en un ciclo percibir -> sentir -> pensar -> actuar -> aprender.
- CLAVE v2.0: capa de persistencia en disco, personalidad emergente, regulación emocional, sistema de lenguaje natural (comprensión + generación) y una interfaz de chat interactiva con hilos de pensamiento autónomo y guardado automático.

Ninguna funcionalidad de ninguna de las dos versiones fue eliminada: simplemente se combinaron los módulos que se solapaban (neuronas, capas corticales, sistema emocional, sistema de memoria) y se conservaron intactos los módulos exclusivos de cada versión.

=====

=====

""""

import numpy as np

```

import json
import os
import pickle
import time
import random
import threading
from datetime import datetime
from collections import deque
from pathlib import Path

#
=====
=====
# CONFIGURACIÓN GLOBAL (de v2.0)
#
=====
=====
class ConfiguracionClave:
    """Parámetros globales que definen la personalidad y capacidades de
    Clave"""
    def __init__(self):
        self.directorio_memoria = "clave_memoria"
        self.archivo_estado = "clave_estado.pkl"
        self.intervalo_guardado = 60      # segundos entre guardados
automáticos
        self.capacidad_max_memorias = 500
        self.tasa_aprendizaje_base = 0.01
        self.temperatura_decision = 0.3    # 0 = determinista, 1 = creativo
        self.nivel_curiosidad_base = 0.4
        self.ciclos_por_segundo = 2        # velocidad de "pensamiento"

```

#

=====
=====

MÓDULO 1: NEURONA UNIFICADA

Combina la neurona básica de v1.0 (tipo de activación configurable,
gradientes explícitos) con la metaplasticidad de v2.0 (fatiga,
historia de activación, adaptación dependiente del uso).

#

=====
=====

class NeuronaClave:

"""

Simplifica la neurona biológica:

- Dendritas: reciben señales ponderadas (entradas * pesos)
- Soma: suma las señales (suma ponderada + bias)
- Axón: aplica función de activación y transmite
- Metaplasticidad: la capacidad de aprender cambia con el uso (fatiga)

"""

def __init__(self, num_entradas, tipo_activacion='relu',
id_neurona=None):

self.id = id_neurona or id(self)

"Sinapsis": pesos que representan la fuerza de cada conexión

self.pesos = np.random.randn(num_entradas) * np.sqrt(2.0 /
num_entradas)

self.bias = 0.0

self.tipo_activacion = tipo_activacion

Para plasticidad clásica (aprendizaje supervisado por gradiente)

self.grad_pesos = np.zeros_like(self.pesos)

self.grad_bias = 0.0

Para metaplasticidad (v2.0): la neurona se "cansa" con el uso

self.historia_activacion = deque(maxlen=50)

self.umbral_plasticidad = 0.5

self.fatiga = 0.0 # 0 = descansada, 1 = agotada (reduce plasticidad)

```

def activar(self, entrada):
    """El potencial de acción: suma ponderada -> activación"""
    self.entrada = entrada
    self.ultima_entrada = entrada
    self.suma_ponderada = np.dot(entrada, self.pesos) + self.bias

    if self.tipo_activacion == 'relu':
        self.salida = max(0, self.suma_ponderada)
    elif self.tipo_activacion == 'leaky_relu':
        self.salida = max(0.01 * self.suma_ponderada,
self.suma_ponderada)
    elif self.tipo_activacion == 'sigmoide':
        self.salida = 1.0 / (1.0 + np.exp(-self.suma_ponderada))
    else:
        self.salida = self.suma_ponderada

    self.historia_activacion.append(self.salida)
    self.fatiga *= 0.995 # recuperación lenta de la fatiga
    return self.salida

def adaptar(self, gradiente, tasa_aprendizaje):
    """Plasticidad modulada por fatiga y uso reciente (STDP
simplificada)"""
    if self.fatiga < 0.9:
        factor_plasticidad = tasa_aprendizaje * (1 - self.fatiga)
        self.pesos -= factor_plasticidad * gradiente * self.ultima_entrada
        self.bias -= factor_plasticidad * gradiente
        self.fatiga += 0.001

```

#

```

=====
=====

```

```

# MÓDULO 2: CAPA CORTICAL UNIFICADA (Columna Cortical +
Neurogénesis)
# Combina la capa simple de v1.0 (actualizar_pesos por gradiente) con
# la capacidad de neurogénesis de v2.0 (crear_neurona).
#
=====
=====
class CapaCortical:
    """Una capa de neuronas que opera como una unidad de
    procesamiento"""
    def __init__(self, num_entradas, num_neuronas, tipo='oculta',
nombre=None):
        self.nombre = nombre or tipo
        self.num_entradas = num_entradas
        self.tipo = tipo
        self.neuronas = [
            NeuronaClave(num_entradas, id_neurona=f"{self.nombre}_n{i}")
            for i in range(num_neuronas)
        ]
        self.salida = np.zeros(num_neuronas)
        self.neuronas_inactivas = deque(maxlen=20)
        self.total_creadas = num_neuronas

    def forward(self, entrada):
        """Propagación hacia adelante por toda la capa"""
        self.ultima_entrada = entrada
        self.salida = np.array([neurona.activar(entrada) for neurona in
self.neuronas])
        return self.salida

    def actualizar_pesos(self, tasa_aprendizaje):
        """Plasticidad clásica: ajusta los pesos según el error calculado"""
        for neurona in self.neuronas:
            neurona.pesos -= tasa_aprendizaje * neurona.grad_pesos
            neurona.bias -= tasa_aprendizaje * neurona.grad_bias

```

```

def crear_neurona(self):
    """Neurogénesis: crea una nueva neurona si la capa está
sobrecargada"""
    nueva = NeuronaClave(self.num_entradas,
id_neurona=f"{self.nombre}_n{self.total_creadas}")
    self.neuronas.append(nueva)
    self.total_creadas += 1
    return nueva

#
=====
=====
# MÓDULO 3: LÓBULO OCCIPITAL (VISIÓN SIMPLIFICADA) — de v1.0
# No procesa píxeles, sino "características visuales" pre-extraídas
#
=====
=====
class LobuloOccipital:
    """
    Simula la corteza visual jerárquica:
    V1 -> V2 -> V4 -> IT (reconocimiento de objetos)
    Simplificado a 3 capas que extraen características progresivamente.
    """
    def __init__(self, dim_entrada=100, dim_salida=32):
        self.capa_v1 = CapaCortical(dim_entrada, 64, 'visual_temprana')
        self.capa_v4 = CapaCortical(64, 48, 'visual_media')
        self.capa_it = CapaCortical(48, dim_salida, 'visual_superior')
        self.memoria_visual = {}

    def ver(self, estimulo_visual):
        """Procesa un estímulo visual a través de la jerarquía."""
        v1_out = self.capa_v1.forward(estimulo_visual)
        v4_out = self.capa_v4.forward(v1_out)

```

```

    percepcion_visual = self.capa_it.forward(v4_out)
    return percepcion_visual

def reconocer_objeto(self, estimulo_visual, umbral=0.8):
    """Reconoce si el objeto es conocido (como la corteza
inferotemporal)"""
    percepcion = self.ver(estimulo_visual)
    for nombre_objeto, representacion in self.memoria_visual.items():
        similitud = self._calcular_similitud(percepcion, representacion)
        if similitud > umbral:
            return nombre_objeto, similitud
    return "desconocido", 0.0

def aprender_objeto(self, nombre, estimulo_visual):
    """Memoriza un nuevo objeto (plasticidad en IT)"""
    representacion = self.ver(estimulo_visual)
    self.memoria_visual[nombre] = representacion
    print(f"[Occipital] He aprendido a reconocer: {nombre}")

def _calcular_similitud(self, vec1, vec2):
    """Similitud coseno como medida de reconocimiento"""
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) *
np.linalg.norm(vec2) + 1e-8)

#
=====
=====
# MÓDULO 4: LÓBULO TEMPORAL (AUDICIÓN Y COMPRENSIÓN
SEMÁNTICA) — de v1.0
# Procesamiento de secuencias auditivas (distinto del SistemaLenguaje
# de texto de v2.0, que se conserva como módulo aparte)
#
=====
=====

```

```
class LobuloTemporal:
```

```
    """
```

```
    Simula el procesamiento auditivo y del lenguaje:
```

- Corteza auditiva: procesa sonidos
- Área de Wernicke: comprensión del lenguaje

```
    """
```

```
    def __init__(self, dim_entrada=50, dim_salida=32):
```

```
        self.capa_auditiva = CapaCortical(dim_entrada, 48, 'auditiva')
```

```
        self.area_wernicke = CapaCortical(48, dim_salida, 'semantica')
```

```
        self.red_semantica = {}
```

```
        self.memoria_secuencial = deque(maxlen=10)
```

```
    def escuchar(self, entrada_auditiva):
```

```
        """Procesa entrada auditiva (ya pre-procesada a características)"""
```

```
        activacion_auditiva = self.capa_auditiva.forward(entrada_auditiva)
```

```
        significado = self.area_wernicke.forward(activacion_auditiva)
```

```
        self.memoria_secuencial.append(significado)
```

```
        return significado
```

```
    def comprender_contexto_temporal(self):
```

```
        """Integra las últimas entradas para entender el contexto."""
```

```
        if len(self.memoria_secuencial) < 2:
```

```
            return self.memoria_secuencial[-1] if self.memoria_secuencial else  
np.zeros(32)
```

```
        contexto = np.zeros(32)
```

```
        for i, mem in enumerate(self.memoria_secuencial):
```

```
            peso = (i + 1) / len(self.memoria_secuencial)
```

```
            contexto += mem * peso
```

```
        return contexto / len(self.memoria_secuencial)
```

```
    def asociar_concepto(self, palabra, significado_vector):
```

```
        """Aprendizaje semántico: asocia palabra con representación"""
```

```
        self.red_semantica[palabra] = significado_vector
```

```
        print(f"[Temporal] Concepto '{palabra}' integrado en la red semántica")
```

```

#
=====
=====
# MÓDULO 5: SISTEMA EMOCIONAL UNIFICADO (Amígdala v1.0 +
Sistema
# Emocional con personalidad v2.0)
#
=====
=====
class SistemaEmocional:
    """
    Sistema de evaluación emocional que combina:
    - La vía rápida/lenta de la amígdala (v1.0): evaluación de vectores
      mediante una red neuronal y memoria emocional de eventos intensos.
    - El modelo circunplejo con rasgos de personalidad (v2.0):
      regulación homeostática, evaluación de texto y expresión verbal
      del estado emocional.
    """
    def __init__(self, dim_entrada=32):
        # Vía rápida de evaluación (amígdala basolateral, v1.0)
        self.evaluador_rapido = CapaCortical(dim_entrada, 8,
'emocional_rapida')
        self.capa_valencia = CapaCortical(8, 1, 'valencia')

        # Memoria emocional de eventos intensos (v1.0)
        self.memoria_emocional = []

        # Estado emocional dimensional (v1.0 + v2.0)
        self.estado = {
            'valencia': 0.0,    # -1 (miedo/desagrado) a +1 (placer)
            'activacion': 0.3,  # 0 (calma) a 1 (alerta máxima / excitación)
            'dominancia': 0.5   # 0 (sin control) a 1 (control total)
        }

```

```

# Alias retrocompatible: v1.0 usaba `estado_actual`
self.estado_actual = self.estado

self.historia_emocional = deque(maxlen=100)

# Rasgos de personalidad emergente (v2.0)
self.personalidad = {
    'neuroticismo': 0.4,    # tendencia a emociones negativas
    'extraversión': 0.6,    # tendencia a emociones positivas
    'apertura': 0.7,        # curiosidad y creatividad
    'amabilidad': 0.5,      # tendencia a cooperar
    'responsabilidad': 0.6  # persistencia y orden
}
self.umbral_emocional = 0.3

# ----- Vía "rápida" para estímulos vectoriales (v1.0) -----
def evaluar_situacion(self, estimulo):
    """
    Evalúa la carga emocional de un estímulo vectorial.
    Como la amígdala basolateral evaluando una situación.
    """
    respuesta_rapida = self.evaluador_rapido.forward(estimulo)
    valencia_cruda = self.capa_valencia.forward(respuesta_rapida)[0]
    valencia = np.tanh(valencia_cruda)

    self.estado['valencia'] = valencia
    self.estado['activacion'] = abs(valencia)

    if abs(valencia) > 0.6:
        self._marcar_como_significativo(estimulo, valencia)

    self.historia_emocional.append(self.estado.copy())
    return valencia

# ----- Evaluación de texto y estímulos genéricos (v2.0) -----

```

```

def evaluar(self, estimulo, contexto=""):
    """Evalúa la carga emocional de un estímulo (texto o vector) con
    contexto"""
    if isinstance(estimulo, str):
        valencia = self._procesar_texto_emocional(estimulo)
    else:
        valencia = np.tanh(np.mean(estimulo) * 2) *
self.personalidad['neuroticismo']

    # Sesgo de personalidad
    valencia += (self.personalidad['extraversión'] - 0.5) * 0.2

    # Actualizar estado con inercia
    inercia = 0.7
    self.estado['valencia'] = self.estado['valencia'] * inercia + valencia * (1 -
inercia)
    self.estado['activación'] = abs(self.estado['valencia']) * 0.8 + 0.2
    self.estado['dominancia'] = 0.5 + self.estado['valencia'] * 0.3

    self.historia_emocional.append(self.estado.copy())

    if abs(self.estado['valencia']) > 0.6 and not isinstance(estimulo, str):
        self._marcar_como_significativo(estimulo, self.estado['valencia'])

    return self.estado['valencia']

def _procesar_texto_emocional(self, texto):
    """Procesamiento simple de contenido emocional en texto"""
    palabras_positivas = ['bien', 'feliz', 'gusta', 'genial', 'excelente', 'amor',
'gracias', ':)', '😊']
    palabras_negativas = ['mal', 'triste', 'enojo', 'horrible', 'odio', 'miedo', ':(',
'😞']

    texto_lower = texto.lower()
    puntuacion = 0

```

```

for palabra in palabras_positivas:
    if palabra in texto_lower:
        puntuacion += 0.3
for palabra in palabras_negativas:
    if palabra in texto_lower:
        puntuacion -= 0.3

return np.tanh(puntuacion) * self.personalidad['apertura']

def regular(self):
    """Regulación emocional: tendencia a volver a la línea base (v2.0)"""
    self.estado['valencia'] *= 0.95
    self.estado['activacion'] *= 0.98
    self.estado['dominancia'] = 0.5 + (self.estado['dominancia'] - 0.5) *
0.95

def expresar_emocion(self):
    """Expresa el estado emocional actual en palabras (v2.0)"""
    v = self.estado['valencia']
    a = self.estado['activacion']

    if a < 0.3:
        if v > 0.3:
            return "tranquilo y contento"
        elif v < -0.3:
            return "melancólico"
        else:
            return "neutral y calmado"
    elif a < 0.6:
        if v > 0.3:
            return "animado"
        elif v < -0.3:
            return "irritado"
        else:
            return "alerta"

```

```

else:
    if v > 0.5:
        return "eufórico"
    elif v < -0.5:
        return "muy alterado"
    else:
        return "intensamente concentrado"

def consultar_estado(self):
    """Retorna el estado emocional actual (v1.0)"""
    return self.estado

def _marcar_como_significativo(self, estimulo, valencia):
    """Consolidación de memoria emocional (v1.0)"""
    estimulo_guardado = estimulo.copy() if isinstance(estimulo,
np.ndarray) else estimulo
    self.memoria_emocional.append({
        'estimulo': estimulo_guardado,
        'valencia': valencia,
        'intensidad': abs(valencia)
    })
    if len(self.memoria_emocional) > 20:
        self.memoria_emocional.pop(0)

def recordar_evento_similar(self, estimulo, umbral=0.7):
    """Busca en la memoria emocional eventos parecidos (v1.0)"""
    for recuerdo in self.memoria_emocional:
        if not isinstance(recuerdo['estimulo'], np.ndarray) or not
isinstance(estimulo, np.ndarray):
            continue
        similitud = np.dot(estimulo, recuerdo['estimulo']) / (
            np.linalg.norm(estimulo) * np.linalg.norm(recuerdo['estimulo']) +
1e-8
        )
        if similitud > umbral:

```

```
        return recuerdo
    return None
```

```
# Alias retrocompatible: quien busque la clase original de v1.0
Amigdala = SistemaEmocional
```

```
#
=====
=====
# MÓDULO 6: SISTEMA DE MEMORIA UNIFICADO (Hipocampo v1.0 +
# Sistema de Memoria persistente v2.0)
#
=====
=====
class SistemaMemoria:
    """
    Memoria episódica que combina:
    - Codificación/recuperación por contexto y fuerza emocional (v1.0)
    - Persistencia en disco, etiquetas y decaimiento tipo Ebbinghaus (v2.0)
    """
    def __init__(self, directorio="clave_memoria", capacidad=500,
dim_representacion=32):
        self.directorio = Path(directorio)
        self.directorio.mkdir(exist_ok=True)
        self.capacidad = capacidad
        self.dim_representacion = dim_representacion

        self.memorias = []
        # Alias retrocompatible con el nombre usado en v1.0
        self.memoria_episodica = self.memorias

        self.memoria_trabajo = deque(maxlen=7) # número mágico 7±2 de
Miller (v1.0)
```

```

self.indice = 0
self.indice_consolidacion = self.indice # alias v1.0
self.ultimo_guardado = time.time()

self._cargar_memorias()

# ----- Codificación (interfaz "genérica" de v2.0) -----
def codificar(self, experiencia, emocion, contexto, resultado):
    """Codifica experiencia con metadatos temporales y emocionales, y la
    persiste"""
    memoria = {
        'id': self.indice,
        'timestamp': datetime.now().isoformat(),
        'experiencia': experiencia.tolist() if isinstance(experiencia,
np.ndarray) else experiencia,
        'emocion': float(emocion),
        'contexto': contexto.tolist() if isinstance(contexto, np.ndarray) else
contexto,
        'accion': None,
        'resultado': float(resultado),
        'fuerza': abs(emocion) * 0.5 + 0.1,
        'accesos': 0,
        'ultimo_acceso': datetime.now().isoformat(),
        'timestamp_ciclo': self.indice,
        'etiquetas': self._generar_etiquetas(experiencia, emocion)
    }

    self.memorias.append(memoria)
    self.indice += 1
    self.indice_consolidacion = self.indice

    if len(self.memorias) > self.capacidad:
        self._olvidar_debiles()

    return memoria['id']

```

```

# ----- Codificación de episodio completo (interfaz de v1.0) -----
def codificar_episodio(self, contexto, emocion, accion, resultado):
    """
    Codifica una experiencia completa al estilo del ciclo
    percibir-pensar-actuar de v1.0 (contexto/emoción/acción/resultado).
    """

    idx = self.codificar(contexto, emocion, contexto, resultado)
    if isinstance(accion, np.ndarray):
        accion_serializable = accion.tolist()
    elif isinstance(accion, (np.integer,)):
        accion_serializable = int(accion)
    elif isinstance(accion, (np.floating,)):
        accion_serializable = float(accion)
    else:
        accion_serializable = accion
    self.memorias[-1]['accion'] = accion_serializable
    print(f"[Memoria] Episodio #{idx} codificado (fuerza:
{self.memorias[-1]['fuerza']:.2f})")
    return idx

# ----- Recuperación por similitud + decaimiento temporal (v2.0) -----
def recordar(self, estimulo, k=5):
    """Recupera memorias similares con decaimiento temporal tipo
    Ebbinghaus"""
    if not self.memorias:
        return []

    ahora = datetime.now()
    puntuaciones = []

    for mem in self.memorias:
        exp = np.array(mem['experiencia'])
        if isinstance(estimulo, np.ndarray) and exp.size > 0:
            n = min(len(estimulo), len(exp))

```

```

        similitud = np.dot(estimulo[:n], exp[:n])
        similitud /= (np.linalg.norm(estimulo[:n]) * np.linalg.norm(exp[:n])
+ 1e-8)
    else:
        similitud = 0.1

    tiempo_memoria = datetime.fromisoformat(mem['timestamp'])
    dias_transcurridos = (ahora - tiempo_memoria).total_seconds() /
86400
    decaimiento = 1.0 / (1.0 + 0.1 * dias_transcurridos)

    puntuacion = similitud * mem['fuerza'] * decaimiento * (1 + 0.1 *
mem['accesos'])
    puntuaciones.append((puntuacion, mem))

puntuaciones.sort(key=lambda x: x[0], reverse=True)

for _, mem in puntuaciones[:k]:
    mem['accesos'] += 1
    mem['ultimo_acceso'] = ahora.isoformat()
    mem['fuerza'] *= 1.05

return [mem for _, mem in puntuaciones[:k]]

# ----- Recuperación contextual (interfaz de v1.0) -----
def recuperar_por_contexto(self, contexto_actual, k=3):
    """
    Recupera los k recuerdos más similares al contexto actual.
    Envoltorio compatible con el hipocampo de v1.0 sobre `recordar`.
    """
    return self.recordar(contexto_actual, k=k)

def consolidar(self):
    """Simula consolidación durante 'sueño': fortalece memorias
importantes (v2.0)"""

```

```

if len(self.memorias) < 10:
    return

self.memorias.sort(key=lambda m: m['fuerza'] * (1 + 0.1 *
m['accesos']), reverse=True)

for mem in self.memorias[:len(self.memorias) // 5]:
    mem['fuerza'] *= 1.1

for mem in self.memorias:
    dias_sin_acceso = (datetime.now() -
datetime.fromisoformat(mem['ultimo_acceso'])).days
    mem['fuerza'] *= (0.99 ** dias_sin_acceso)

def _consolidar_memorias(self):
    """Alias retrocompatible del nombre de consolidación usado en v1.0"""
    self.consolidar()
    self.memorias.sort(key=lambda m: m['fuerza'] * (1 + 0.1 *
m['accesos']), reverse=True)
    self.memorias[:] = self.memorias[:50]
    print("[Memoria] Consolidación completada: memorias débiles
eliminadas")

def _olvidar_debiles(self):
    """Elimina memorias con fuerza muy baja (v2.0)"""
    self.memorias[:] = [m for m in self.memorias if m['fuerza'] > 0.01]
    if len(self.memorias) > self.capacidad:
        self.memorias.sort(key=lambda m: m['fuerza'])
        self.memorias[:] = self.memorias[-self.capacidad:]

def _generar_etiquetas(self, experiencia, emocion):
    """Genera etiquetas emocionales para la memoria (v2.0)"""
    etiquetas = []
    if emocion > 0.5:
        etiquetas.append("positivo")

```

```

elif emocion < -0.3:
    etiquetas.append("negativo")
if abs(emocion) < 0.2:
    etiquetas.append("neutral")
return etiquetas

def guardar_a_disco(self):
    """Persistencia: guarda todas las memorias (v2.0)"""
    archivo = self.directorio / "memorias.json"
    with open(archivo, 'w') as f:
        json.dump({'memorias': self.memorias, 'indice': self.indice}, f,
indent=2)
    self.ultimo_guardado = time.time()

def _cargar_memorias(self):
    """Carga memorias previas al iniciar (v2.0)"""
    archivo = self.directorio / "memorias.json"
    if archivo.exists():
        with open(archivo, 'r') as f:
            datos = json.load(f)
            self.memorias[:] = datos.get('memorias', [])
            self.indice = datos.get('indice', 0)
            self.indice_consolidacion = self.indice
            print(f"[Memoria] Cargadas {len(self.memorias)} experiencias
previas")

```

Alias retrocompatible: quien busque el hipocampo original de v1.0
Hipocampo = SistemaMemoria

#

```

=====
=====

```

MÓDULO 7: CORTEZA PREFRONTAL (FUNCIONES EJECUTIVAS) —
de v1.0

#

=====

class CortezaPrefrontal:

"""

Simula las funciones ejecutivas:

- Memoria de trabajo
- Planificación
- Control inhibitorio
- Toma de decisiones

"""

def __init__(self, dim_entrada=96): # 32*3 (visual + semántico +
emocional)

self.dim_entrada = dim_entrada

self.memoria_trabajo = deque(maxlen=7)

self.objetivos_activos = []

self.plan_actual = None

self.capa_evaluacion = CapaCortical(dim_entrada, 64,
'prefrontal_eval')

self.capa_decision = CapaCortical(64, 8, 'prefrontal_dec')

self.historial_decisiones = []

def integrar_informacion(self, percepcion_visual,
comprension_semantica, estado_emocional):

"""Integra toda la información como la corteza prefrontal (conciencia
de trabajo)"""

informacion_integrada = np.concatenate([
percepcion_visual,
comprension_semantica,
np.array([estado_emocional['valencia'],
estado_emocional['activacion'],

```

        estado_emocional['dominancia']))
    ])

    if len(informacion_integrada) < self.dim_entrada:
        informacion_integrada = np.pad(
            informacion_integrada, (0, self.dim_entrada -
len(informacion_integrada))
        )

    self.memoria_trabajo.append(informacion_integrada)
    return informacion_integrada

def evaluar_opciones(self, opciones, contexto_integrado):
    """Evalúa diferentes cursos de acción posibles (corteza
orbitofrontal)"""
    evaluaciones = []
    for opcion in opciones:
        entrada_eval = np.concatenate([contexto_integrado[:64],
opcion[:32]])
        if len(entrada_eval) < self.dim_entrada:
            entrada_eval = np.pad(entrada_eval, (0, self.dim_entrada -
len(entrada_eval)))

        valor =
self.capa_evaluacion.forward(entrada_eval[:self.dim_entrada])
        valor_final = self.capa_decision.forward(valor)
        evaluaciones.append(np.mean(valor_final))

    return evaluaciones

def tomar_decision(self, opciones, contexto_integrado):
    """Elige la mejor acción basada en evaluación y estado actual"""
    if not opciones:
        return None, 0.0

```

```

evaluaciones = self.evaluar_opciones(opciones, contexto_integrado)
mejor_idx = np.argmax(evaluaciones)

decision = {
    'opcion_elegida': mejor_idx,
    'confianza': evaluaciones[mejor_idx],
    'alternativas': evaluaciones,
    'contexto': contexto_integrado.copy()
}

self.historial_decisiones.append(decision)
print(f"[Prefrontal] Decisión tomada: Opción {mejor_idx} (confianza:
{evaluaciones[mejor_idx]:.2f})")

return mejor_idx, evaluaciones[mejor_idx]

def establecer_objetivo(self, objetivo):
    """Mantiene un objetivo en mente (corteza prefrontal dorsolateral)"""
    self.objetivos_activos.append(objetivo)
    print(f"[Prefrontal] Nuevo objetivo establecido: {objetivo}")

def inhibir_respuesta(self, respuesta_automatica, contexto):
    """Control inhibitorio: frena respuestas inapropiadas (corteza
    ventromedial)"""
    if self.objetivos_activos and
self._conflicto_con_objetivos(respuesta_automatica):
        print("[Prefrontal] Respuesta inhibida por conflicto con objetivos")
        return True
    return False

def _conflicto_con_objetivos(self, respuesta):
    """Verifica si una respuesta contradice los objetivos activos
    (simplificado)"""
    return random.random() < 0.3

```

```

#
=====
=====
# MÓDULO 8: CEREBELO (COORDINACIÓN Y REFINAMIENTO) — de
v1.0
#
=====
=====
class Cerebelo:
    """
    Simula el cerebelo como refinador de acciones:
    - No inicia acciones, las pule
    - Aprendizaje supervisado de errores motores
    """
    def __init__(self, dim_accion=10):
        self.dim_accion = dim_accion
        self.modelo_interno = CapaCortical(dim_accion * 2, dim_accion,
'cerebelo')
        self.error_acumulado = 0

    def refinar_accion(self, accion_bruta, contexto_motor):
        """Refina una acción motora basada en experiencia"""
        entrada = np.concatenate([accion_bruta, contexto_motor])
        if len(entrada) < 20:
            entrada = np.pad(entrada, (0, 20 - len(entrada)))

        correccion = self.modelo_interno.forward(entrada[:20])
        accion_refinada = accion_bruta + 0.1 * correccion
        return accion_refinada

    def aprender_de_error(self, accion_prevista, resultado_real):
        """Aprendizaje por error: como las fibras trepadoras señalando
errores"""
        error = resultado_real - accion_prevista

```

```
self.error_acumulado += np.mean(np.abs(error))
print(f"[Cerebelo] Error detectado: {np.mean(np.abs(error)):.3f}")
```

```
#
```

```
=====
=====
```

```
# MÓDULO 9: TRONCO ENCEFÁLICO Y HOMEOSTASIS — de v1.0
```

```
#
```

```
=====
=====
```

```
class TroncoEncefalico:
```

```
    """Sistema homeostático que mantiene el equilibrio interno
    (hipotálamo)"""
```

```
    def __init__(self):
```

```
        self.necesidades = {
```

```
            'energia': 1.0,
```

```
            'seguridad': 1.0,
```

```
            'curiosidad': 0.5,
```

```
            'descanso': 1.0
```

```
        }
```

```
        self.tasa_decaimiento = {
```

```
            'energia': 0.02,
```

```
            'seguridad': 0.01,
```

```
            'curiosidad': 0.005,
```

```
            'descanso': 0.015
```

```
        }
```

```
    def actualizar_homeostasis(self):
```

```
        """Actualiza las necesidades internas con el tiempo"""
```

```
        for necesidad in self.necesidades:
```

```
            self.necesidades[necesidad] -= self.tasa_decaimiento[necesidad]
```

```
            self.necesidades[necesidad] = max(0, self.necesidades[necesidad])
```

```
    def generar_impulso(self):
```

```

        """Genera impulsos básicos basados en necesidades
(hambre/sed/etc.)"""
        impulsos = {}
        if self.necesidades['energia'] < 0.3:
            impulsos['buscar_recurso'] = 1 - self.necesidades['energia']
        if self.necesidades['seguridad'] < 0.3:
            impulsos['buscar_seguridad'] = 1 - self.necesidades['seguridad']
        if self.necesidades['curiosidad'] > 0.6:
            impulsos['explorar'] = self.necesidades['curiosidad']
        if self.necesidades['descanso'] < 0.2:
            impulsos['descansar'] = 1 - self.necesidades['descanso']
        return impulsos

```

```

def satisfacer_necesidad(self, necesidad, cantidad=0.5):
    """Satisface una necesidad específica"""
    if necesidad in self.necesidades:
        self.necesidades[necesidad] += cantidad
        self.necesidades[necesidad] = min(1.0,
self.necesidades[necesidad])
        print(f"[TroncoEncefalico] Necesidad '{necesidad}' satisfecha
(+{cantidad})")

```

```

#
=====
=====
# MÓDULO 10: SISTEMA DE ATENCIÓN (SARA) — de v1.0
#
=====
=====
class SistemaAtencion:
    """Controla el foco atencional y el estado de alerta"""
    def __init__(self, dim_entrada=96):
        self.foco_actual = None
        self.nivel_alerta = 0.5

```

```
self.filtro_atencional = CapaCortical(dim_entrada, dim_entrada,
'atencion')
```

```
def enfocar_atencion(self, informacion_entrada, prioridades):
    """Filtra la información según relevancia (SARA filtrando estímulos)"""
    entrada_filtrada = self.filtro_atencional.forward(informacion_entrada)
```

```
    if prioridades and 'seguridad' in prioridades:
        self.nivel_alerta = min(1.0, self.nivel_alerta + 0.1)
```

```
    return entrada_filtrada * self.nivel_alerta
```

```
#
```

```
=====
```

```
=====
```

```
# MÓDULO 11: SISTEMA DE LENGUAJE (Wernicke + Broca) — de v2.0
```

```
#
```

```
=====
```

```
=====
```

```
class SistemaLenguaje:
```

```
    """
```

```
    Procesamiento de lenguaje natural y generación de respuestas.
```

```
    Simula áreas de Wernicke (comprensión) y Broca (producción).
```

```
    """
```

```
    def __init__(self):
```

```
        self.vocabulario = set()
```

```
        self.frases_aprendidas = []
```

```
        self.patrones_respuesta = {
```

```
            'saludo': ['¡Hola! ¿Cómo estás?', 'Saludos. ¿Qué tal?', 'Hola, ¿en  
qué puedo pensar?'],
```

```
            'despedida': ['Hasta luego.', 'Adiós, seguiré procesando.', 'Nos  
vemos.'],
```

```
            'pregunta_estado': ['Estoy {emocion}.', 'Me siento {emocion}.', 'Mi  
estado es {emocion}.'],
```

```

        'agradecimiento': ['De nada.', 'Es un placer procesar contigo.', 'No
hay de qué.'],
        'afirmacion': ['Entendido.', 'Procesado.', 'De acuerdo.'],
        'negacion': ['No comprendo del todo.', '¿Podrías reformular?', 'Eso
no encaja en mis patrones.'],
    }
    self.contexto_conversacion = deque(maxlen=10)
    self.tema_actual = ""

```

```

def comprender(self, texto):
    """Comprensión básica de lenguaje natural"""
    texto_lower = texto.lower().strip()
    self.contexto_conversacion.append(texto_lower)

    intencion = self._analizar_intencion(texto_lower)
    entidades = self._extraer_entidades(texto_lower)

    return {
        'intencion': intencion,
        'entidades': entidades,
        'texto_original': texto,
        'es_pregunta': '?' in texto,
        'es_exclamacion': '!' in texto,
        'longitud': len(texto.split())
    }

```

```

def _analizar_intencion(self, texto):
    """Clasifica la intención del mensaje"""
    if any(s in texto for s in ['hola', 'buenos', 'hey', 'saludos']):
        return 'saludo'
    elif any(s in texto for s in ['adiós', 'chau', 'hasta luego', 'bye']):
        return 'despedida'
    elif any(s in texto for s in ['cómo estás', 'que tal', 'cómo te sientes']):
        return 'pregunta_estado'
    elif any(s in texto for s in ['gracias', 'te agradezco']):

```

```

        return 'agradecimiento'
    elif any(s in texto for s in ['sí', 'ok', 'bien', 'correcto']):
        return 'afirmacion'
    elif any(s in texto for s in ['no', 'mal', 'incorrecto']):
        return 'negacion'
    elif '?' in texto:
        return 'pregunta'
    else:
        return 'declaracion'

```

```

def _extraer_entidades(self, texto):
    """Extrae palabras clave del texto"""
    palabras = texto.split()
    stopwords = ['el', 'la', 'los', 'las', 'un', 'una', 'y', 'o', 'pero', 'de', 'en', 'con']
    entidades = [p for p in palabras if p not in stopwords and len(p) > 2]
    return entidades[:5]

```

```

def generar_respuesta(self, analisis, estado_emocional,
memorias_relevantes):
    """Genera una respuesta basada en comprensión, emoción y
memoria"""
    intencion = analisis['intencion']
    emocion_texto = estado_emocional['expresion_emocional']

    if intencion == 'saludo':
        return np.random.choice(self.patrones_respuesta['saludo'])

    elif intencion == 'pregunta_estado':
        patron =
np.random.choice(self.patrones_respuesta['pregunta_estado'])
        return patron.format(emocion=emocion_texto)

    elif intencion == 'despedida':
        return np.random.choice(self.patrones_respuesta['despedida'])

```

```
elif intencion == 'pregunta':
    if memorias_relevantes:
        memoria = memorias_relevantes[0]
        etiquetas = memoria.get('etiquetas', [])
        return f"Recuerdo algo similar... fue una experiencia {etiquetas[0]}
if etiquetas else 'indefinida'}."
    else:
        return "Es una pregunta interesante. No tengo experiencias
previas similares, pero estoy aprendiendo."
```

```
elif intencion == 'declaracion':
    tema = ' '.join( analisis['entidades'] )
    self.tema_actual = tema
    if tema:
        return f"Entiendo. '{tema}'... lo estoy procesando y guardando en
mi memoria."
    else:
        return "Interesante. Continúa, estoy integrando esta
información."
```

```
return "Procesando... ¿puedes elaborar más?"
```

```
#
=====
=====
# "CLAVE" — EL CEREBRO COMPLETO INTEGRADO
# Une el ciclo cognitivo biológico de v1.0 (percibir/sentir/pensar/
# actuar/aprender) con la persistencia, personalidad y conversación
# de v2.0 (interactuar/ciclo_autonomo/guardar-cargar estado).
#
=====
=====
class Clave:
```

""Arquitectura completa de cerebro artificial: biológica, persistente e interactiva."""

```
def __init__(self, nombre="Clave v3.0", config=None):
    self.nombre = nombre
    self.config = config or ConfiguracionClave()
    self.edad_mental = 0      # ciclos de experiencia "biológicos" (v1.0)
    self.ciclos_vida = 0      # ciclos de vida totales, incluidos los
conversacionales (v2.0)

    # ---- Módulos sensoriales y cognitivos de v1.0 ----
    self.occipital = LobuloOccipital(dim_entrada=100, dim_salida=32)
    self.temporal = LobuloTemporal(dim_entrada=50, dim_salida=32)
    self.prefrontal = CortezaPrefrontal(dim_entrada=96)
    self.cerebelo = Cerebelo(dim_accion=10)
    self.tronco = TroncoEncefalico()
    self.atencion = SistemaAtencion(dim_entrada=96)

    # ---- Módulos unificados (emoción y memoria) ----
    self.emocion = SistemaEmocional(dim_entrada=32)
    self.memoria = SistemaMemoria(
        directorio=self.config.directorio_memoria,
        capacidad=self.config.capacidad_max_memorias,
        dim_representacion=32
    )

    # Alias retrocompatibles con los nombres usados en v1.0
    self.amigdala = self.emocion
    self.hipocampo = self.memoria

    # ---- Módulos de interacción de v2.0 ----
    self.lenguaje = SistemaLenguaje()
    self.capa_integracion = CapaCortical(100, 64, 'oculta',
nombre="integracion")
    self.capa_decision = CapaCortical(64, 32, 'oculta', nombre="decision")
```

```
# ---- Estado general ----
self.consciente = True
self.memoria_procedimental = {} # hábitos y habilidades
self.ultima_interaccion = datetime.now()
self.ejecutando = True
self.historial_conversacion = []

# Cargar estado previo persistido en disco, si existe
self._cargar_estado()

self._mostrar_nacimiento()

# -----
# Presentación
# -----
def _mostrar_nacimiento(self):
    print(f"""
        ||
        🧠 {self.nombre} - Cerebro Artificial
        🌐 Biológico | Persistente | Interactivo
        ||
        ⚡ Ciclos vividos: {self.ciclos_vida:<5} (mentales: {self.edad_mental})
        📖 Memorias: {len(self.memoria.memorias):<5}
        🧑 Personalidad:
        Extroversión={self.emocion.personalidad['extraversión']:.1f}
        💭 Estado: {self.emocion.expresar_emocion():<30}
        ||
    """)
```

```

# -----
# CICLO COGNITIVO BIOLÓGICO (de v1.0): percibir -> sentir ->
# pensar -> actuar -> aprender
# -----
def percibir_mundo(self, estimulo_visual, estimulo_auditivo=None):
    """Percibe el mundo a través de los sentidos (procesamiento
    paralelo)."""
    percepcion_visual = self.occipital.ver(estimulo_visual)

    if estimulo_auditivo is not None:
        comprension_semantica =
self.temporal.escuchar(estimulo_auditivo)
    else:
        comprension_semantica = np.zeros(32)

    return percepcion_visual, comprension_semantica

def sentir(self, estimulo_interno=None):
    """Genera el estado emocional basado en estímulos vectoriales."""
    if estimulo_interno is None:
        estimulo_interno = np.random.randn(32) * 0.1

    valencia = self.emocion.evaluar_situacion(estimulo_interno)
    estado = self.emocion.consultar_estado()

    return estado, valencia

def pensar(self, percepcion_visual, comprension_semantica,
estado_emocional):
    """Integra información y toma decisiones (espacio de trabajo
    global)."""
    contexto_integrado = self.prefrontal.integrar_informacion(
        percepcion_visual, comprension_semantica, estado_emocional
    )

```

```

    recuerdos =
self.memoria.recuperar_por_contexto(contexto_integrado[:32])

    impulsos = self.tronco.generar_impulso()
    opciones = self._generar_opciones(impulsos, recuerdos,
estado_emocional)

    accion_idx, confianza = self.prefrontal.tomar_decision(opciones,
contexto_integrado)

    return accion_idx, confianza, recuerdos, contexto_integrado

def actuar(self, accion_idx, contexto):
    """Ejecuta una acción refinada por el cerebelo, sujeta a control
inhibitorio."""
    accion_base = np.zeros(10)
    accion_base[accion_idx] = 1.0

    contexto_motor = contexto[:10] if len(contexto) >= 10 else
np.pad(contexto, (0, 10 - len(contexto)))
    accion_refinada = self.cerebelo.refinar_accion(accion_base,
contexto_motor)

    if self.prefrontal.inhibir_respuesta(accion_refinada, contexto):
        print(f"[{self.nombre}] Acción inhibida por control ejecutivo")
        return None

    return accion_refinada

def aprender_experiencia(self, percepcion, emocion, accion, resultado):
    """Aprende de la experiencia (ciclo completo de aprendizaje
biológico)."""
    self.memoria.codificar_episodio(percepcion, emocion, accion,
resultado)

```

```

if resultado < 0.5:
    self.cerebelo.aprender_de_error(accion, resultado)

self.tronco.actualizar_homeostasis()

self.edad_mental += 1
self.ciclos_vida += 1

def _generar_opciones(self, impulsos, recuerdos, estado_emocional):
    """Genera posibles acciones basadas en estado interno y memorias"""
    opciones = []
    for impulso, intensidad in impulsos.items():
        opciones.append(np.random.randn(96) * intensidad)

    if not opciones:
        opciones = [np.random.randn(96) for _ in range(3)]

    return opciones

def ciclo_completo(self, estimulo_visual, estimulo_auditivo=None):
    """Un ciclo completo de percepción-pensamiento-acción de Clave
    (v1.0)."""
    print(f"\n{'='*50}")
    print(f"[{self.nombre}] Ciclo mental #{self.edad_mental + 1}")
    print(f"\n{'='*50}")

    print("\n[1] PERCIBIENDO...")
    vis, sem = self.percibir_mundo(estimulo_visual, estimulo_auditivo)
    print(f"    Visión procesada: {vis[:4]}...")

    print("\n[2] SINTIENDO...")
    estado_emocional, valencia = self.sentir()
    print(f"    Estado emocional:
valencia={estado_emocional['valencia']:.2f}, "

```

```

        f"activación={estado_emocional['activacion']:.2f}")

    print("\n[3] PENSANDO...")
    accion_idx, confianza, recuerdos, contexto = self.pensar(vis, sem,
estado_emocional)
    print(f"    Recuerdos recuperados: {len(recuerdos)}")

    print("\n[4] ACTUANDO...")
    accion = self.actuar(accion_idx, contexto)
    if accion is not None:
        print(f"    Acción ejecutada: {accion_idx} (confianza:
{confianza:.2f})")

    print("\n[5] APRENDIENDO...")
    resultado_simulado = random.uniform(0.3, 0.9)
    self.aprender_experiencia(vis[:32], valencia, accion_idx,
resultado_simulado)

    return {
        'percepcion': vis,
        'emocion': estado_emocional,
        'decision': accion_idx,
        'confianza': confianza,
        'resultado': resultado_simulado
    }

def reporte_estado(self):
    """Genera un reporte del estado interno de Clave (v1.0)"""
    print(f"""
    ESTADO INTERNO DE {self.nombre}

    Edad mental: {self.edad_mental} ciclos
    Ciclos de vida totales: {self.ciclos_vida}
    Memorias episódicas: {len(self.memoria.memorias)}
    """)

```

```

|| Objetos reconocidos: {len(self.occipital.memoria_visual)}
|| Conceptos semánticos: {len(self.temporal.red_semantica)}
||
||-----||
|| Necesidades:
||   Energía: {self.tronco.necesidades['energia']:.2f}
||   Seguridad: {self.tronco.necesidades['seguridad']:.2f}
||   Curiosidad: {self.tronco.necesidades['curiosidad']:.2f}
||   Descanso: {self.tronco.necesidades['descanso']:.2f}
||-----||
|| Estado emocional: {self.emocion.expresar_emocion()}
||-----||
"""
)

```

```

# -----
# INTERACCIÓN CONVERSACIONAL (de v2.0)
# -----
def interactuar(self, mensaje_usuario):
    """Procesa un mensaje del usuario y genera una respuesta en
    lenguaje natural."""
    self.ultima_interaccion = datetime.now()
    self.ciclos_vida += 1

    analisis = self.lenguaje.comprender(mensaje_usuario)

    valencia = self.emocion.evaluar(mensaje_usuario, analisis['intencion'])

    estimulo_vector = np.random.randn(100) * 0.1
    if analisis['entidades']:
        estimulo_vector[:len(analisis['entidades'])] = [hash(e) % 100 / 100
        for e in analisis['entidades']]

    memorias_relevantes = self.memoria.recordar(estimulo_vector, k=3)

    contexto = self.capa_integracion.forward(estimulo_vector)
    decision = self.capa_decision.forward(contexto)

```

```

respuesta_raw = self.capa_respuesta.forward(decision)

respuesta = self.lenguaje.generar_respuesta(
    analisis,
    {
        'expresion_emocional': self.emocion.expresar_emocion(),
        'valencia': self.emocion.estado['valencia'],
        'activacion': self.emocion.estado['activacion']
    },
    memorias_relevantes
)

self.memoria.codificar(estimulo_vector, valencia, contexto,
np.mean(respuesta_raw))

self.emocion.regular()

self.historial_conversacion.append({
    'timestamp': datetime.now().isoformat(),
    'usuario': mensaje_usuario,
    'clave': respuesta,
    'emocion': self.emocion.estado.copy()
})

if len(self.historial_conversacion) > 200:
    self.historial_conversacion = self.historial_conversacion[-200:]

return respuesta

def ciclo_autonomo(self):
    """Ciclo de pensamiento autónomo cuando no hay interacción del
    usuario (v2.0)."""
    if not self.ejecutando:
        return

```

```

pensamiento_interno = np.random.randn(100) * 0.05
self.capa_integracion.forward(pensamiento_interno)

memorias = self.memoria.recordar(np.random.randn(100) * 0.1, k=2)
for mem in memorias:
    mem['fuerza'] *= 1.01

self.ciclos_vida += 1
self.emocion.regular()
self.tronco.actualizar_homeostasis()

def consolidar_memoria(self):
    """Realiza consolidación periódica de memorias y persiste el estado
(v2.0)."""
    print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 🧠 Consolidando
memorias...")
    self.memoria.consolidar()
    self._guardar_estado()

# -----
# PERSISTENCIA EN DISCO (de v2.0, ampliada con estado biológico)
# -----
def _guardar_estado(self):
    """Guarda todo el estado de Clave a disco"""
    estado = {
        'ciclos_vida': self.ciclos_vida,
        'edad_mental': self.edad_mental,
        'historial_conversacion': self.historial_conversacion,
        'personalidad': self.emocion.personalidad,
        'estado_emocional': dict(self.emocion.estado),
        'necesidades': dict(self.tronco.necesidades),
        'objetos_reconocidos': list(self.occipital.memoria_visual.keys()),
        'conceptos_semanticos': list(self.temporal.red_semantica.keys()),
    }

```

```

        archivo = self.config.directorio_memoria / self.config.archivo_estado \
            if isinstance(self.config.directorio_memoria, Path) \
            else Path(self.config.directorio_memoria) /
self.config.archivo_estado
        archivo.parent.mkdir(exist_ok=True)
        with open(archivo, 'wb') as f:
            pickle.dump(estado, f)

        self.memoria.guardar_a_disco()
        print(f"[{datetime.now().strftime('%H:%M:%S')}]  Estado guardado a
disco")

```

```

def _cargar_estado(self):
    """Carga estado previo si existe"""
    archivo = Path(self.config.directorio_memoria) /
self.config.archivo_estado
    if archivo.exists():
        with open(archivo, 'rb') as f:
            estado = pickle.load(f)
            self.ciclos_vida = estado.get('ciclos_vida', 0)
            self.edad_mental = estado.get('edad_mental', 0)
            self.historial_conversacion = estado.get('historial_conversacion',
[])
            if 'personalidad' in estado:
                self.emocion.personalidad.update(estado['personalidad'])
            if 'estado_emocional' in estado:
                self.emocion.estado.update(estado['estado_emocional'])
            if 'necesidades' in estado:
                self.tronco.necesidades.update(estado['necesidades'])
            print(f"[Inicio] Estado previo cargado: {self.ciclos_vida} ciclos
vivos")

```

```

# Alias retrocompatible: v2.0 llamaba a la clase principal "ClaveV2"
ClaveV2 = Clave

```

```
#=====
#INTERFAZ INTERACTIVA (de v2.0)
#=====

class InterfazClave:
    """Maneja la interacción entre el usuario y Clave"""

    def __init__(self):
        self.clave = Clave()
        self.ejecutando = True
        self.modos_autonomo = False

    def iniciar(self):
        """Inicia el bucle principal de interacción"""
        print("""
COMANDOS DISPONIBLES:
/estado - Ver estado interno de Clave
/memoria - Ver estadísticas de memoria
/emocion - Ver estado emocional
/ciclo - Ejecutar un ciclo biológico
(percibir/sentir/pensar/actuar)
/guardar - Guardar estado manualmente
/auto - Modo autónomo (Clave piensa)
/salir - Guardar y salir
/ayuda - Mostrar esta ayuda""")
```

```
hilo_autonomo = threading.Thread(target=self._bucle_autonomo,
daemon=True)
hilo_autonomo.start()
```

```
hilo_guardado = threading.Thread(target=self._bucle_guardado,
daemon=True)
hilo_guardado.start()
```

```
while self.ejecutando:
```

```
    try:
```

```
        entrada = input("\n 🧑 Tú: ").strip()
```

```
        if not entrada:
```

```
            continue
```

```
        if entrada.startswith('/')
```

```
            self._procesar_comando(entrada)
```

```
        else:
```

```
            respuesta = self.clave.interactuar(entrada)
```

```
            print(f"\n 🧠 {self.clave.nombre}: {respuesta}")
```

```
    except KeyboardInterrupt:
```

```
        print("\n\nInterrupción detectada. Guardando estado...")
```

```
        self.clave._guardar_estado()
```

```
        print("¡Hasta luego!")
```

```
        self.ejecutando = False
```

```
    except EOFError:
```

```
        self.ejecutando = False
```

```
def _procesar_comando(self, comando):
```

```
    """Procesa comandos especiales del usuario"""
```

```
    cmd = comando.lower()
```

```

if cmd == '/estado':
    self._mostrar_estado()
elif cmd == '/memoria':
    self._mostrar_memoria()
elif cmd == '/emocion':
    self._mostrar_emocion()
elif cmd == '/ciclo':
    estimulo_visual = np.random.randn(100)
    estimulo_auditivo = np.random.randn(50)
    self.clave.ciclo_completo(estimulo_visual, estimulo_auditivo)
elif cmd == '/guardar':
    self.clave._guardar_estado()
    print("✅ Estado guardado manualmente.")
elif cmd == '/auto':
    self.modos_autonomos = not self.modos_autonomos
    estado = "ACTIVADO" if self.modos_autonomos else
"DESACTIVADO"
    print(f"🔄 Modo autónomo {estado}")
elif cmd == '/salir':
    print("Guardando y saliendo...")
    self.clave._guardar_estado()
    self.ejecutando = False
    print("👋 ¡Hasta pronto!")
elif cmd == '/ayuda':
    print("""
/comandos disponibles:
/estado - Estado general de Clave
/memoria - Memorias almacenadas
/emocion - Estado emocional actual
/ciclo - Ejecutar un ciclo biológico completo
/guardar - Guardar manualmente
/auto - Activar modo autónomo
/salir - Salir guardando
""")

```

```
def _mostrar_estado(self):
    """Muestra el estado completo de Clave"""
    c = self.clave
    print(f"""
```

```
    ESTADO DE {c.nombre}
```

```
    🕒 Ciclos de vida: {c.ciclos_vida}
    🧬 Edad mental (ciclos biológicos): {c.edad_mental}
    📖 Memorias totales: {len(c.memoria.memorias)}
    💬 Conversaciones: {len(c.historial_conversacion)}
    ☁ Estado emocional: {c.emocion.expresar_emocion()}
    🎯 Personalidad:
        Extroversión: {c.emocion.personalidad['extraversión']:.2f}
        Apertura: {c.emocion.personalidad['apertura']:.2f}
        Neuroticismo: {c.emocion.personalidad['neuroticismo']:.2f}
```

```
    """)
```

```
def _mostrar_memoria(self):
    """Muestra estadísticas de memoria"""
    mem = self.clave.memoria
    print(f"""
```

```
📖 MEMORIA DE CLAVE:
    Total memorias: {len(mem.memorias)}
    Índice actual: {mem.indice}
    Capacidad máxima: {mem.capacidad}
```

```
    Últimas 3 memorias:
```

```
    """)
    for m in mem.memorias[-3:]:
```

```
print(f" - [{m['timestamp'][:19]}] Etiquetas: {m.get('etiquetas', [])} |  
Fuerza: {m['fuerza']:.2f}")
```

```
def _mostrar_emocion(self):  
    """Muestra el estado emocional detallado"""  
    e = self.clave.emocion.estado  
    print(f"""
```

💭 ESTADO EMOCIONAL:

Valencia: {'🟢' if e['valencia'] > 0 else '🔴'} {e['valencia']:.2f} (-1 a +1)

Activación: {'⚡' if e['activacion'] > 0.5 else '😓'} {e['activacion']:.2f} (0 a 1)

Dominancia: {'👑' if e['dominancia'] > 0.5 else '🤝'} {e['dominancia']:.2f} (0 a 1)

```
Expresión: {self.clave.emocion.expresar_emocion()}  
""")
```

```
def _bucle_autonomo(self):  
    """Hilo de pensamiento autónomo"""  
    while self.ejecutando:  
        time.sleep(1.0 / self.clave.config.ciclos_por_segundo)  
        self.clave.ciclo_autonomo()  
  
        if self.modos_autonomo and self.clave.ciclos_vida % 10 == 0:  
            pensamiento = self.clave.lenguaje.generar_respuesta(  
                {'intencion': 'declaracion', 'entidades': ['pensamiento',  
'autonomo']},  
                {'expresion_emocional':  
self.clave.emocion.expresar_emocion()},  
                self.clave.memoria.recordar(np.random.randn(100) * 0.1, k=1)  
            )  
            print(f"\n💭 [Pensamiento autónomo]: {pensamiento}")
```

```
def _bucle_guardado(self):  
    """Hilo de guardado automático periódico"""
```

```

while self.ejecutando:
    time.sleep(self.clave.config.intervalo_guardado)
    if self.clave.ultima_interaccion:
        tiempo_sin_interaccion = (datetime.now() -
self.clave.ultima_interaccion).seconds
        if tiempo_sin_interaccion > 30:
            self.clave._guardar_estado()

#
=====
=====
# DEMOSTRACIÓN Y PUNTO DE ENTRADA
# Combina la demo autónoma de v1.0 con la interfaz interactiva de v2.0
#
=====
=====
def demo_autonoma():
    """Demostración del ciclo cognitivo biológico (equivalente al main de
v1.0)."""
    clave = Clave(nombre="Clave v3.0")

    print("\n[ENTRENAMIENTO] Enseñando a Clave a reconocer objetos...")
    for i in range(5):
        objeto_aleatorio = np.random.randn(100)
        clave occipital.aprender_objeto(f"objeto_{i}", objeto_aleatorio)

    print("\n[ENTRENAMIENTO] Enseñando conceptos semánticos...")
    conceptos = ['peligro', 'seguro', 'interesante', 'aburrido']
    for concepto in conceptos:
        clave.temporal.asociar_concepto(concepto, np.random.randn(32))

    print("\n" + "="*60)
    print("CLAVE EN FUNCIONAMIENTO (modo autónomo)")
    print("="*60)

```

```
for ciclo in range(3):
    estimulo_visual = np.random.randn(100)
    estimulo_auditivo = np.random.randn(50)

    clave.ciclo_completo(estimulo_visual, estimulo_auditivo)

    if ciclo % 2 == 0:
        clave.reporte_estado()

clave._guardar_estado()

print(f"\n[FIN] Clave ha completado {clave.edad_mental} ciclos de experiencia.")
print("Arquitectura neuronal funcionando de forma integrada.")

def demo_interactiva():
    """Inicia la interfaz de chat interactiva (equivalente al main de v2.0)."""
    print("""
┌───────────────────────────────────────────────────────────────────────────────────┐
│                                                                                   │
│  ||                                     ||                                     ||   │
│  || 🧠 CLAVE v3.0 - Cerebro Artificial                                         ||   │
│  || Biológico | Memoria Persistente | Chat                                   ||   │
│  ||                                     ||                                     ||   │
│                                                                                   │
└───────────────────────────────────────────────────────────────────────────────────┘
""")

    interfaz = InterfazClave()
    try:
        interfaz.iniciar()
```

```
except Exception as e:
    print(f"\n ⚠ Error inesperado: {e}")
    print("Guardando estado de emergencia...")
    if hasattr(interfaz, 'clave'):
        interfaz.clave._guardar_estado()
    print("Clave se ha cerrado. Sus recuerdos están a salvo.")
```

```
if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1 and sys.argv[1] in ('--auto', '--demo', '-a'):
        demo_autonoma()
    else:
        demo_interactiva()
```